

Product Requirements Document (PRD)

Edge Rule Engine

Status: Draft

Version: 1.0

Target: Edge Device

Primary Technology: Go

Database: SQLite

Expression Engine: Expr

Initial Action Target: File

Future Action Target: HTTP/API

1. Overview

The Edge Rule Engine is a lightweight, event-driven rules-processing service that runs locally on an edge device.

The engine receives real-time data from a Computer Vision (CV) pipeline in two forms:

1. **State data** — a camera snapshot sent approximately every 5 seconds.
2. **Event data** — an event notification sent only when a defined event occurs.

Separately, rules are provided by an external **UI / Rule API**. The Edge Rule Engine stores those rules and evaluates them against incoming CV data.

When a rule's conditions are satisfied, the engine executes the configured action.

The initial MVP will write the resulting action to a file. The action mechanism will be abstracted so that it can later be replaced or extended with HTTP/API calls.

The engine is intended to be lightweight enough for deployment on edge devices and should support approximately **15 concurrently active task rules**, including multiple rules operating on the same camera.

2. Problem Statement

The CV pipeline produces raw observations, but business behavior requires additional logic such as:

- detecting high occupancy,
- identifying a queue that remains above a threshold,
- detecting prolonged dwell,
- counting events over a time window,
- combining multiple conditions,

- preventing repeated actions while a condition remains true,
- triggering an action when a condition becomes satisfied.

These calculations should not be implemented directly in the CV pipeline.

The Rule Engine will provide a separate business-logic layer that consumes CV observations and converts them into actions.

3. Goals

3.1 Primary Goals

The system must:

- Run as a lightweight service on an edge device.
- Be implemented primarily in Go.
- Receive state data over HTTP.
- Receive event data over HTTP.
- Receive/create/update task rules through an external Rule API.
- Persist state, events, and rules locally.
- Maintain an in-memory working window for real-time evaluation.
- Support state-based rules.
- Support event-based rules.
- Support rolling time windows.
- Support conditions that must remain true for a specified duration.
- Support logical conditions such as `AND` and `OR`.
- Support entity-level evaluation using `trackId`.
- Prevent duplicate or excessive action execution through trigger semantics and cooldowns.
- Provide a pluggable action execution layer.
- Provide file-based action execution for MVP.

3.2 Secondary Goals

The design should allow future support for:

- HTTP/webhook actions.
 - Additional event types.
 - Additional aggregations.
 - More complex rules.
 - Additional edge-device integrations.
 - Operational metrics and health monitoring.
-

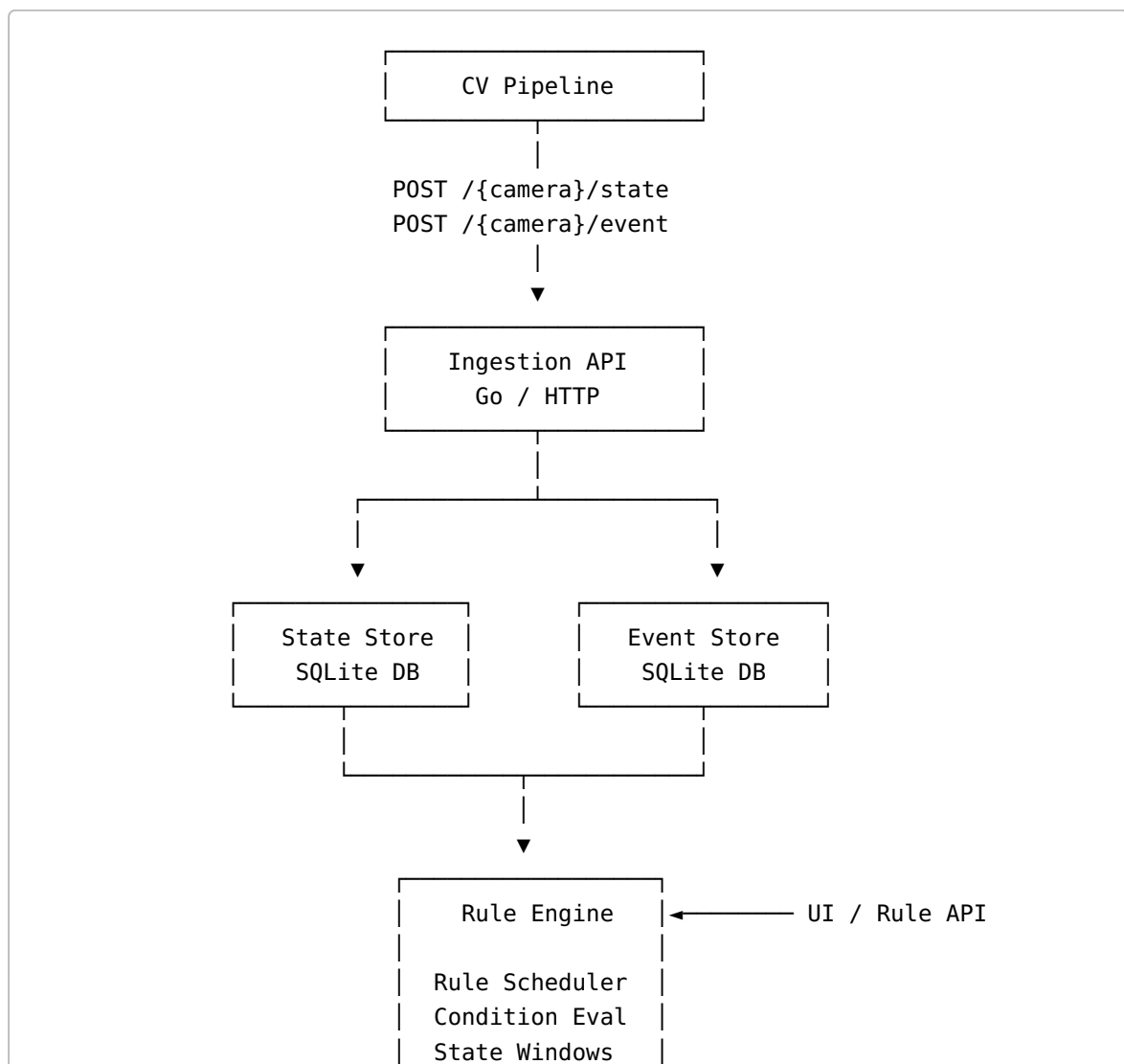
4. Non-Goals

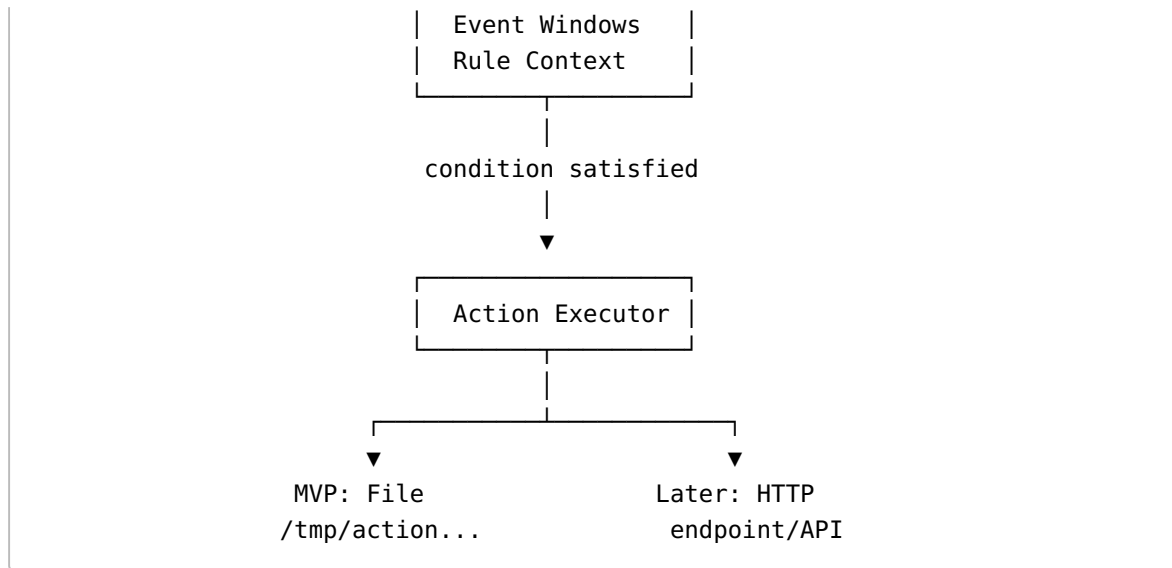
The MVP will not:

- Implement a UI.
- Implement CV detection or tracking.
- Perform object detection itself.
- Replace the CV pipeline.
- Require Kafka, RabbitMQ, Redis, or another external message broker.
- Require PostgreSQL/MySQL.
- Execute arbitrary Go code as a rule.
- Allow expressions to directly execute shell commands, HTTP requests, or database operations.
- Make business-rule decisions inside the CV pipeline.

The external UI/Rule API is responsible for creating and managing rules.

5. High-Level Architecture





6. Core Architectural Decision

The system has **three distinct input responsibilities**.

6.1 CV State

Represents:

"What the camera sees now."

State is delivered approximately every 5 seconds.

Used for:

- occupancy,
- staff count,
- current entities,
- dwell,
- current ROI conditions,
- sustained conditions.

6.2 CV Event

Represents:

"Something happened."

Events are sent only when an event occurs.

Used for:

- entrance/event counting,

- line crossing,
- exit events,
- event-based triggers,
- rolling event windows.

6.3 Rule Input

Represents:

"What business behavior should be applied to the CV data."

Rules are provided by an external UI / Rule API.

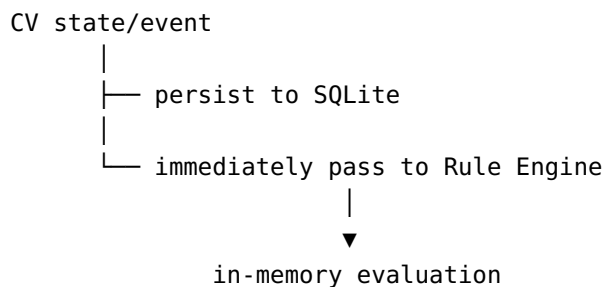
The Rule Engine stores and evaluates these rules.

7. Source of Truth vs Real-Time Processing

SQLite is the persistent source of truth for historical information.

However, the Rule Engine should **not query SQLite for every 5-second state update**.

The intended flow is:



SQLite is used for:

- persistence,
- recovery,
- historical inspection,
- rule storage,
- action/execution records if required.

In-memory state is used for:

- latest state,
- rolling windows,
- active rule context,
- timers,
- trigger state,
- cooldown state.

This keeps the edge system lightweight and minimizes unnecessary database reads.

8. CV State Contract

The current state payload is considered sufficient for the initial rule set.

Example:

```
{
  "schemaVersion": "1.3",
  "kind": "state",
  "emittedAt": 1753796476,
  "locationName": "site-01",
  "location": "6012sadamads",
  "cameraName": "cam-3",
  "camera": "6004hjk23k42k",
  "regions": [
    {
      "roi": "entrance",
      "roiId": "60dhkjsa",
      "occupancy": {
        "total": 2,
        "person": 1,
        "vehicle": 1
      },
      "precalc": {
        "dwell": {
          "person": {
            "avg": 8,
            "min": 8,
            "max": 8
          },
          "vehicle": {
            "avg": 40,
            "min": 40,
            "max": 40
          }
        }
      },
      "entities": [
        {
          "entityType": "person",
          "trackId": "p-1001",
          "dwellTime": 8,
          "confidence": 0.94,
          "attributes": {
            "age": "18-35",
            "gender": "Male",
```

```

        "attention": "Yes"
    }
}
]
}
]
}

```

8.1 Confirmed CV Semantics

The CV team has confirmed:

- `trackId` remains stable for an entity during tracking.
- `trackId` is unique within a camera/tracking session.
- `eventId` is unique.
- `emittedAt` has consistent/global timestamp semantics.
- `staffCount` is reliably populated for applicable ROIs.
- `precalc.dwell.*.max` represents the maximum dwell time among all entities of that type currently present in the ROI.

For example:

```

Person dwell times:
5s, 8s, 12s

avg = 8.33s
min = 5s
max = 12s

```

8.2 `trackId`

`trackId` is an important entity identifier.

The Rule Engine can use it to identify that:

```

14:00:00 → p-1001
14:00:05 → p-1001
14:00:10 → p-1001
14:00:15 → p-1001

```

represents the same tracked entity.

The engine must not assume that `trackId` is globally unique forever. It is unique within the camera/tracking-session scope defined by the CV system.

9. CV Event Contract

Example:

```
{
  "kind": "line_crossed",
  "eventId": "evt-000342",
  "emittedAt": 1753796500,
  "locationName": "location_name",
  "location": "57tggvjgjj",
  "roi": "line",
  "roiId": "6767hgjhjh",
  "cameraName": "cam-3",
  "camera": "67fjhjgg",
  "entity": {
    "entityType": "person",
    "trackId": "p-1001",
    "dwellTime": null,
    "confidence": 0.94,
    "attributes": {
      "age": "18-35",
      "gender": "Male",
      "attention": "Yes"
    }
  }
}
```

Another example:

```
{
  "kind": "exit",
  "eventId": "evt-000343",
  "emittedAt": 1753796500,
  "locationName": "location_name",
  "location": "57tggvjgjj",
  "roi": "entrance",
  "roiId": "6767hgjhjh",
  "cameraName": "cam-3",
  "camera": "67fjhjgg",
  "entity": {
    "entityType": "person",
    "trackId": "p-1001",
    "dwellTime": 32,
    "confidence": 0.94,
    "attributes": {
      "age": "18-35",
      "gender": "Male",
      "attention": "Yes"
    }
  }
}
```



```
}  
}  
}
```

Event Design Decision

The Rule Engine does **not require line-crossing direction** for the current requirements.

The event type/ROI provides the relevant event context.

10. Rule Input / Rule API

The engine does not implement a UI.

Rules are received through an external UI / Rule API.

Example conceptual operations:

```
POST    /api/v1/rules  
GET     /api/v1/rules  
GET     /api/v1/rules/{ruleId}  
PUT     /api/v1/rules/{ruleId}  
DELETE  /api/v1/rules/{ruleId}
```

The exact API contract may be finalized separately.

The Rule Engine must:

1. Validate the rule.
 2. Persist the rule.
 3. Compile/prepare the expression.
 4. Register or update the active rule in memory.
 5. Apply the updated rule without requiring a process restart where practical.
-

11. Rule Model

A rule should describe:

- identity,
- scope,
- source,
- condition,
- temporal behavior,
- trigger behavior,

- action.

Example:

```
{
  "id": "rule-001",
  "name": "High Queue Without Staff",
  "enabled": true,

  "scope": {
    "cameraId": "cam-3",
    "roiId": "checkout-queue"
  },

  "source": "state",

  "condition": {
    "expression": "occupancy.person > 5"
  },

  "trigger": {
    "mode": "rising",
    "cooldownSeconds": 300
  },

  "action": {
    "type": "notify_manager"
  }
}
```

12. Expression Engine

The project will use **Expr** as the expression evaluation layer.

Expr will be responsible for evaluating business conditions such as:

```
occupancy.person > 5
```

```
occupancy.person > 5 && staffCount == 0
```

```
dwelling.person.max > 120
```

```
window.entranceCount >= 10
```

The Rule Engine will remain responsible for:

- ingestion,
- state/event windows,
- timers,
- deduplication,
- trigger state,
- cooldowns,
- scheduling,
- action execution,
- persistence.

12.1 Important Decision

Expr is **not** the entire rule engine.

It is only the **condition evaluation layer**.

The temporal and operational logic remains in Go.

13. Why Expr Is Used

Using Expr avoids implementing a custom expression parser in Go.

The Rule Engine can expose a controlled evaluation context such as:

```
occupancy.person  
occupancy.vehicle  
occupancy.total  
staffCount  
  
dwell.person.avg  
dwell.person.min  
dwell.person.max  
  
entities  
window.entranceCount
```

Example:

```
occupancy.person > 5 && staffCount == 0
```

The engine creates the appropriate context and Expr returns the condition result.

14. Rule Sources

Each rule must explicitly identify its data source:

```
state
event
```

This makes evaluation deterministic.

A state rule is evaluated when relevant state arrives.

An event rule is evaluated when a relevant event arrives.

15. Temporal Rule Types

The engine must support three primary temporal patterns.

15.1 Instantaneous Condition

Example:

```
occupancy.person > 5
```

Evaluate the latest applicable state.

15.2 Sustained Condition

Example:

```
occupancy.person > 5 for 3 minutes
```

The condition must remain true continuously for 3 minutes.

Example:

```
14:00:00 → 6 > 5 → timer starts
14:00:05 → 6 > 5
14:00:10 → 7 > 5
...
14:03:00 → 6 > 5
```

↓
Trigger

If the condition becomes false before the duration is reached, the timer resets.

Example:

14:01:40 → 4 ≤ 5
↓
reset

15.3 Rolling Window

Example:

entrance count ≥ 10 within 120 seconds

The engine maintains a sliding time window.

For events:

14:00:10
14:00:25
14:00:48
14:01:05
14:01:40

The engine removes entries older than the configured window and evaluates the remaining events.

16. State Window

State arrives approximately every 5 seconds.

For a 120-second state window:

120 / 5 = approximately 24 snapshots

The engine can keep the working state window entirely in memory.

A rolling state window should:

1. Add the new state.
2. Remove expired states.
3. Update relevant aggregates/context.
4. Evaluate applicable rules.

The engine should not query SQLite for every evaluation.

17. Event Window

Event windows work similarly but contain event records rather than 5-second state snapshots.

The engine should maintain an in-memory time-window structure for relevant events.

Example:

```
event received
  ↓
append to event window
  ↓
remove events older than window
  ↓
calculate count
  ↓
evaluate rule
```

18. Track ID and Counting

Because the same entity appears across multiple state snapshots, the engine must not treat every snapshot occurrence as a new person/vehicle.

Example:

```
14:00:00 p-1001
14:00:05 p-1001
14:00:10 p-1001
14:00:15 p-1001
```

This represents:

```
1 unique tracked entity
```

not:

```
4 people
```

Where state-window calculations require unique entities, the engine should deduplicate using the tracked entity identity.

Conceptually:

```
camera + entityType + trackId
```

with tracking-session semantics defined by the CV pipeline.

19. Trigger Semantics

The engine must distinguish between:

```
condition is true
```

and:

```
condition became true
```

The default behavior should be **rising-edge triggering**.

Example:

```
14:00:00 → occupancy = 6 → FALSE → TRUE → trigger
14:00:05 → occupancy = 7 → TRUE → no trigger
14:00:10 → occupancy = 8 → TRUE → no trigger
14:00:15 → occupancy = 9 → TRUE → no trigger
14:00:20 → occupancy = 4 → TRUE → FALSE
14:00:25 → occupancy = 6 → FALSE → TRUE → trigger
```

This prevents an action from executing every 5 seconds while the condition remains true.

20. Cooldown

Rules should support a configurable cooldown.

Example:

```
cooldown = 300 seconds
```

Once triggered, the action cannot execute again until the cooldown expires.

Cooldown is independent from condition evaluation.

This is useful for notifications and repeated screen changes.

21. Action Executor

The action layer must be abstracted.

Conceptually:

```
type ActionExecutor interface {  
    Execute(ctx context.Context, action Action) error  
}
```

MVP

The initial implementation writes an action record to a file.

Example:

```
/tmp/action-<timestamp>-<ruleId>.json
```

Example output:

```
{  
  "ruleId": "rule-001",  
  "triggeredAt": 1753796500,  
  "action": "notify_manager",  
  "cameraId": "cam-3",  
  "roiId": "checkout-queue"  
}
```

Future

The same interface can support:

HTTP
Webhook
Screen control
Notification systems
Other edge integrations

The Rule Engine should not need to change its condition logic when the action mechanism changes.

22. Initial Rule Set

The MVP must support the following rules.

Rule 1 — Entrance Count

Definition

For {CameraID},
IF entrance count $\geq$ N within 120 seconds
THEN Trigger.

Source:

EVENT

Logic:

```
filter applicable camera/event context
  ↓
collect relevant entrance events
  ↓
maintain rolling 120-second window
  ↓
count qualifying events
  ↓
count  $\geq$  N
  ↓
Trigger
```

This rule should not calculate entrance count by summing occupancy from 5-second state snapshots.

Rule 2 — Checkout Queue Threshold

Definition

```
For {CameraID}, for {ROI-ID},  
IF checkout queue occupancy.person > N  
THEN Trigger.
```

Source:

```
STATE
```

Example expression:

```
occupancy.person > N
```

Rule 3 — Maximum Dwell

Definition

```
For {CameraID}, for {ROI-ID},  
IF dwell.max > 2 minutes  
THEN Trigger.
```

Source:

```
STATE
```

For person dwell:

```
dwell.person.max > 120
```

Based on the confirmed CV semantics, this means:

At least one currently present person has a dwell time greater than 120 seconds.

Rule 4 — Queue Sustained for 3 Minutes

Definition

```
For {CameraID}, for {ROI-ID},  
IF queue occupancy.person > N for 3 minutes  
THEN Trigger.
```

Source:

```
STATE
```

Example:

```
occupancy.person > N
```

Temporal requirement:

```
for = 3m
```

The timer resets whenever the condition becomes false.

Rule 5 — Occupancy Threshold

Definition

```
For {CameraID}, for {ROI-ID},  
IF occupancy.person > N  
THEN Trigger.
```

Source:

```
STATE
```

Example:

```
occupancy.person > N
```

Rule 6 — Occupancy With No Staff

Definition

```
For {CameraID}, for {ROI-ID},  
IF occupancy.person > N  
AND no staff detected  
THEN Trigger.
```

Source:

STATE

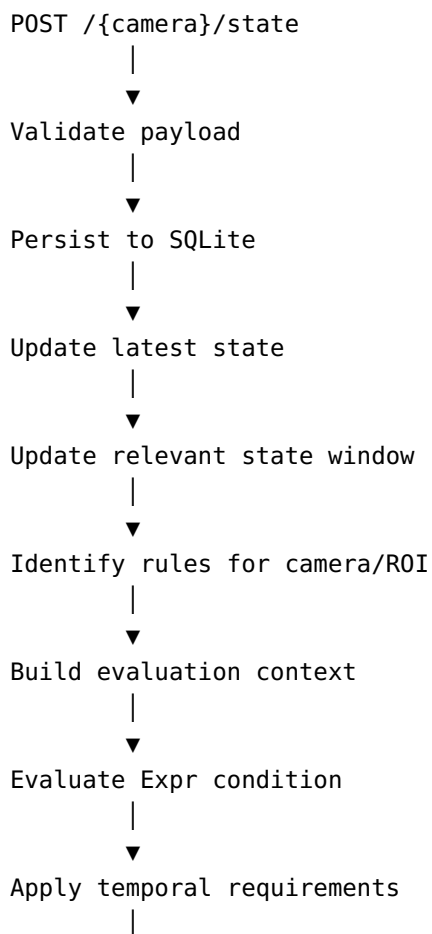
Example:

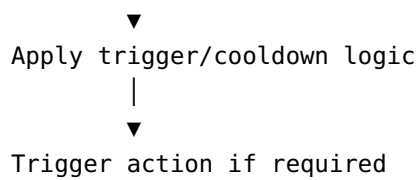
```
occupancy.person > N && staffCount == 0
```

`staffCount` is supplied by the CV pipeline and is considered reliable for applicable ROIs.

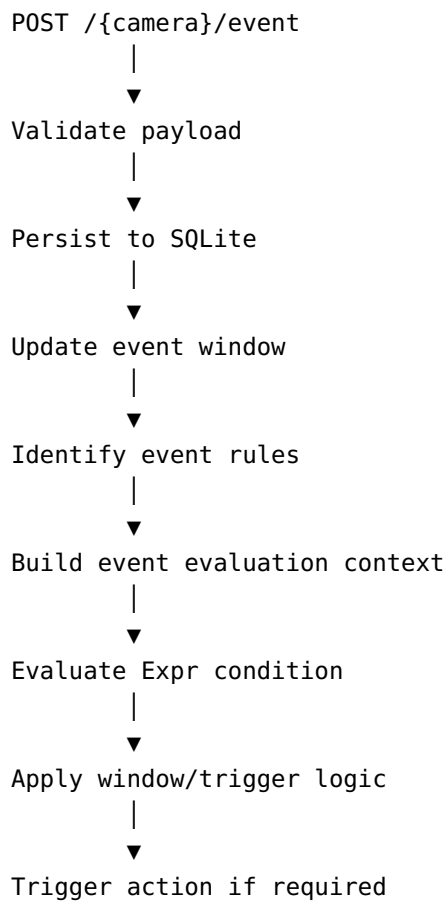
23. Rule Evaluation Flow

For a state update:



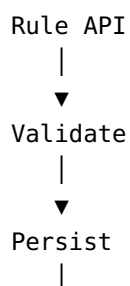


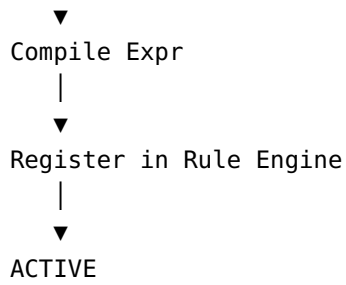
For an event:



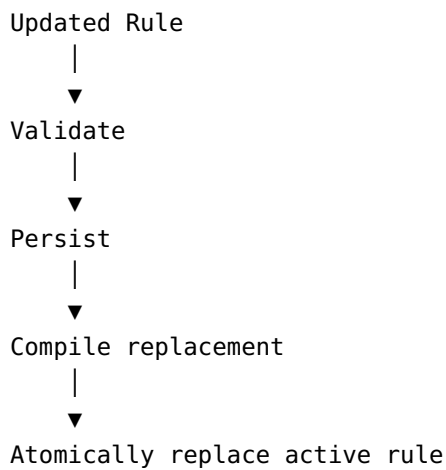
24. Rule Registration and Lifecycle

When a new rule arrives:





When a rule is updated:



When disabled:

```
enabled = false
```

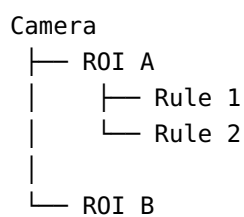
The rule remains persisted but should not be evaluated.

25. Rule Selection and Efficiency

The engine should avoid evaluating every rule against every incoming message.

Rules should be indexed by relevant scope.

Conceptually:



```
|— Rule 3
|— Rule 4
```

For event rules, the engine can additionally index by relevant event type.

This is an optimization and organization mechanism rather than a strict scalability requirement, since the expected active-rule count is approximately 15.

26. Concurrency

The system must support approximately:

```
15 active rules concurrently
```

Rules may belong to:

- different cameras,
- different ROIs,
- the same camera,
- the same ROI.

The implementation should use Go concurrency primitives with controlled execution rather than unlimited goroutine creation.

The engine must ensure that concurrent state/event handling does not corrupt:

- rule state,
- rolling windows,
- timers,
- cooldowns,
- execution records.

27. Persistence

SQLite should be used for local persistence.

At minimum, the database should contain:

```
task_rules
states
events
```

Optional/likely useful:

```
rule_executions
actions
```

Suggested conceptual schema

task_rules

```
id
name
enabled
camera_id
roi_id
source
rule_json
created_at
updated_at
```

states

```
id
camera_id
emitted_at
payload
```

events

```
id
event_id
camera_id
kind
emitted_at
payload
```

rule_executions

```
id
rule_id
triggered_at
status
action_type
context
error
```

The MVP can retain raw JSON payloads rather than fully normalizing all nested CV data.

28. Error Handling

The engine must handle:

- malformed JSON,
- unsupported schema versions,
- missing required fields,
- unknown camera/ROI references,
- invalid rules,
- invalid expressions,
- action execution failures,
- duplicate events,
- repeated event delivery,
- out-of-order timestamps where applicable.

A malformed request should not crash the service.

An invalid rule should not be activated.

An action failure should be recorded and logged.

29. Duplicate Handling

Because `eventId` is unique, the ingestion layer should be able to detect duplicate event delivery.

Duplicate events should not be counted twice.

For state data, the implementation may use:

- `stateId` if introduced later by the CV contract, or
- timestamp/sequence semantics if available.

A dedicated `stateId` is **not required for the current MVP**, but could be introduced later if duplicate/out-of-order state handling becomes necessary.

30. Timestamp Handling

`emittedAt` is the common/global timestamp supplied by the CV pipeline.

The Rule Engine should use it for:

- rolling windows,
- sustained-duration rules,
- event age,
- expiration,
- event/state ordering.

The engine should not depend exclusively on local receipt time for business-rule calculations.

Receipt time may still be useful for diagnostics and operational metrics.

31. State vs Event Decision Matrix

Requirement	State	Event
Current occupancy	✓	✗
Current staff count	✓	✗
Current dwell	✓	✗
Entity presence	✓	✗
Sustained occupancy	✓	✗
Queue threshold	✓	✗
Entrance count	✗	✓
Line crossing count	✗	✓
Event window	✗	✓
Track identity	✓	✓

This separation is a core architectural decision.

32. API Responsibilities

CV Ingestion API

Responsible for:

```
POST /{camera}/state
POST /{camera}/event
```

Responsibilities:

- request validation,
- persistence,
- forwarding to real-time evaluation.

Rule API

Responsible for:

```
POST    /rules
GET      /rules
PUT      /rules/{id}
DELETE  /rules/{id}
```

Responsibilities:

- rule validation,
- rule persistence,
- rule activation/deactivation,
- notification/update of the Rule Engine.

33. Recommended Internal Components

The Go implementation should be logically separated into:

```
api/
  state handler
  event handler
  rule handler

model/
  state
  event
  rule
  action

store/
  SQLite
  state repository
  event repository
  rule repository

engine/
  engine
  evaluator
  scheduler
  trigger manager

window/
  state window
  event window
  aggregations

action/
  executor interface
  file executor
```

```
future HTTP executor

cache/
  latest state
  active rules
```

This separation should remain lightweight and should not result in unnecessary microservices.

34. Configuration

The service should support a small configuration file or equivalent environment configuration.

Potential settings:

```
HTTP listen address
SQLite database path
Action directory
log level
state retention
event retention
maximum active rules
default cooldown
```

Example:

```
server:
  address: ":8080"

database:
  path: "/var/lib/wobot/rule-engine/engine.db"

actions:
  directory: "/var/lib/wobot/rule-engine/actions"

logging:
  level: "info"
```

Exact configuration format can be finalized during implementation.

35. Observability

The MVP should at minimum provide:

/health

and structured logs for:

- state received,
- event received,
- rule created/updated/disabled,
- rule matched,
- action triggered,
- action failed,
- invalid rule,
- invalid payload.

Future versions can expose metrics such as:

```
states_received_total
events_received_total
rules_active
rules_evaluated_total
rules_triggered_total
actions_executed_total
actions_failed_total
```

36. Security

The MVP should assume the service is deployed in a controlled edge environment.

Nevertheless:

- validate all incoming JSON,
- do not allow arbitrary code execution through Expr,
- do not expose dangerous functions to Expr,
- keep action execution outside the expression evaluator,
- restrict HTTP action destinations when HTTP actions are introduced,
- avoid allowing expressions to directly access SQLite or the OS.

The expression should remain conceptually:

Input Data → Expression → Result

Actions should remain:

Expression Result → Action Executor

37. Performance Requirements

The engine should be lightweight relative to an edge device.

Expected workload:

- state updates approximately every 5 seconds per camera,
- event updates only when an event occurs,
- approximately 15 active rules,
- potentially multiple rules for the same camera.

Performance should be achieved primarily through:

- in-memory latest state,
- in-memory rolling windows,
- indexed rule selection,
- precompiled expressions,
- SQLite for persistence rather than repeated runtime queries.

The engine should not require an external cache or message broker for MVP.

38. MVP Scope

The first implementation should support:

Data

- HTTP state ingestion.
- HTTP event ingestion.
- SQLite persistence.
- In-memory latest state.
- In-memory state windows.
- In-memory event windows.

Rules

- state rules,
- event rules,
- camera scope,
- ROI scope,
- numeric comparisons,
- logical `AND`,
- logical `OR`,
- entity access,
- `trackId`-based deduplication,
- rolling time windows,
- sustained conditions,
- rising-edge trigger,
- cooldown.

Expression Engine

- Expr integration.
- Expression validation at rule creation/update.
- Precompiled expressions.

Actions

- File-based action executor.

Operations

- health endpoint,
 - structured logging,
 - basic execution/error recording.
-

39. Post-MVP

Potential later capabilities:

- HTTP/webhook actions.
 - Screen-specific actions.
 - Richer aggregations.
 - More event types.
 - Rule templates.
 - Rule simulation/testing.
 - Rule execution history API.
 - Metrics dashboard.
 - Rule replay against historical data.
 - More advanced entity correlation.
 - Camera/ROI metadata configuration.
 - Additional action providers.
-

40. Future Example: Product Dwell

A future rule may require:

```
IF person dwell > 2 minutes in display-zone  
THEN show the corresponding product promotion  
on the nearest screen.
```

The current entity data provides:

```
trackId  
dwellTime
```

entityType
attributes

This is sufficient to identify a tracked person and dwell duration.

However, identifying the **specific product being interacted with** requires an additional CV relationship/object identifier if the CV pipeline does not already provide one.

This should therefore be treated as a future data-contract requirement rather than a mandatory MVP change.

41. Key Decisions

The following decisions are considered settled for the MVP:

1. **Go** will be the primary implementation language.
 2. **SQLite** will be used for local persistence.
 3. **Expr** will be used for dynamic business-condition evaluation.
 4. Expr will **not** implement temporal scheduling or actions.
 5. State data is sent approximately every **5 seconds**.
 6. Event data is sent **only when events occur**.
 7. State and event processing are separate concepts.
 8. State windows will be maintained **in memory**.
 9. Event windows will be maintained **in memory**.
 10. SQLite will not be queried repeatedly for every rule evaluation.
 11. `trackId` is already available and will be used for entity correlation/deduplication.
 12. `trackId` is stable during entity tracking and unique within the camera/tracking-session context.
 13. `eventId` is unique.
 14. `emittedAt` is the consistent/global CV timestamp.
 15. Line-crossing direction is **not required** by the current rule set.
 16. `staffCount` can be trusted for applicable ROIs.
 17. `dwell.max` represents the maximum dwell among currently present entities of that type.
 18. Rules will use declarative configuration plus Expr conditions.
 19. Rules will support instantaneous, sustained-duration, and rolling-window semantics.
 20. Default rule triggering should be rising-edge based.
 21. Rules should support cooldowns.
 22. Actions will be abstracted behind an executor interface.
 23. MVP actions will write files.
 24. HTTP/API actions will be added later.
 25. No UI will be implemented inside the Edge Rule Engine.
 26. The external UI / Rule API is responsible for sending/managing rules.
 27. No external queue/broker is required for MVP.
 28. Approximately 15 active rules is the initial concurrency target.
-

42. Acceptance Criteria

The MVP is considered complete when the following can be demonstrated:

Data ingestion

- A valid state payload can be accepted over HTTP.
- A valid event payload can be accepted over HTTP.
- Data is persisted to SQLite.
- Incoming data reaches the rule engine without requiring a subsequent database query.

Rule management

- A rule can be created.
- Invalid Expr expressions are rejected.
- Rules persist across process restart.
- Rules can be enabled/disabled.
- Updated rules become active without unnecessary service restart.

State rules

The engine correctly evaluates:

```
occupancy.person > N
```

```
occupancy.person > N && staffCount == 0
```

```
dwelling.person.max > 120
```

Sustained rules

The engine correctly evaluates:

```
occupancy.person > N for 3 minutes
```

and resets the timer if the condition becomes false.

Event rules

The engine correctly evaluates:

```
entrance count >= N within 120 seconds
```

and removes expired events from the rolling window.

Entity handling

Repeated state observations for the same `trackId` do not incorrectly inflate unique-entity counts.

Trigger behavior

A continuously true condition does not repeatedly trigger on every 5-second state update when using rising-edge semantics.

Actions

A satisfied rule produces the expected MVP action file.

Reliability

A malformed input or invalid rule does not crash the service.

43. Summary

The Edge Rule Engine is intentionally designed as a **small single-node edge service** rather than a distributed rules platform.

The architecture separates:

```
CV observations
  ↓
Ingestion
  ↓
Persistence + real-time context
  ↓
Rule Engine
  ↓
Expr condition evaluation
  ↓
Temporal/trigger logic
  ↓
Action Executor
```

The most important design principle is:

CV provides observations; the Rule Engine provides business logic.

State data represents the current situation and is used for occupancy, dwell, staff, and sustained conditions.

Event data represents occurrences and is used for event counting and rolling event windows.

`trackId` provides continuity of entity identity across state snapshots.

Expr provides the flexible condition language, while Go handles the real-time windowing, scheduling, deduplication, trigger state, persistence, and action execution.

This keeps the MVP lightweight, understandable, and extensible without introducing infrastructure that the edge deployment does not need.